



ELEC3609/9609 - Week 1 Lab

01 Structured Planning and System Design for Web Applications

This lab introduces foundational planning techniques for web application development. You will explore how to define system requirements, sketch early design ideas, and model your application's database structure. These practices support clearer team communication and more maintainable software systems.

Individual Task: Individual Task 1 - System Specification and Data Modelling

Lab Resources: IEEE Standard Documents - **IEEE 830** and **IEEE 1233**.

Learning Objectives

After completing this lab, students should be able to:

- Identify and write well-formed functional requirements.
- Create effective wireframes and early design prototypes.
- Model entity relationships using Crow's Foot notation in ER diagrams.

Chapter 1. Software Design and Planning

1.1 Introduction

In software engineering, formal documents like the System Requirements Specification (SyRS) and the Software Requirements Specification (SRS) are used to define what a system should do before implementation begins. A SyRS outlines the broader system context and objectives, while an SRS focuses more narrowly on the software's expected behaviour and features.

While this course does not require students to produce full SyRS or SRS documents, a ci

understanding of functional requirements, the specific tasks or services the system must perform, is essential. Writing well-formed functional requirements is a key skill that supports effective system design and project planning.

This lab introduces practical techniques to identify and express functional requirements for your team's web application. We'll also draw on principles from industry-standard documentation practices to help you avoid vague or incomplete requirements and ensure your designs can be validated and implemented effectively.

1.2 Well-Formed Requirements: Capability, Condition, Constraint

Defined in IEEE 1233-1998 (SyRS), a well-formed requirement includes:

- **Capability:** The core functionality the system must perform.
- **Condition:** Preconditions or input required to trigger that capability.
- **Constraint:** Boundaries or non-functional requirements the system must follow.

Example (User Authentication):

- **Capability:** The system shall authenticate users securely before granting access to sensitive information.
- **Condition:** Users must provide valid credentials (username and password).
- **Constraint:** Passwords must be stored using industry-standard hashing algorithms with a salt.

WARNING

Writing *only* what "you think is important" is **not** enough. Good requirements are **verifiable, complete, and aligned** with a structured approach, no matter which documentation you use.

1.3 Comparing SRS and SyRS

Aspect	SRS (IEEE 830)	SyRS (IEEE 1233)

Aspect	SRS (IEEE 830)	SyRS (IEEE 1233)
Scope	Software system	Entire system (software + hardware + process)
Audience	Developers, testers, project managers	Stakeholders, engineers, end-users
Purpose	Blueprint for coding, testing, maintenance	Defines full system architecture and interactions
Relation	More detailed, component-level	High-level, integrative

**TIP**

There is no single “best” documentation method. What matters is whether you **cover the necessary information** to ensure your system design is valid, complete, and ready for implementation.

1.4 Functional Requirements (IEEE 830-1998)

In Assignment 1, you are expected to extract and write **functional requirements** for your team’s web application. These define what the software must do, the required behaviours in response to inputs and events.

Below is the official definition adapted from **IEEE 830-1998, Section 5.3.2**:

Functional requirements should define the fundamental actions that must take place in the software in accepting and processing the inputs and in processing and generating the outputs. These are generally listed as “shall” statements starting with “The system shall...”

These include:

- a) Validity checks on the inputs
- b) Exact sequence of operations
- c) Responses to abnormal situations, including:
 - i. Overflow



- ii. Communication facilities
- iii. Error handling and recovery
- d) Effect of parameters
- e) Relationship of outputs to inputs, including:
 - i. Input/output sequences
 - ii. Formulas for input to output conversion

It may be appropriate to partition the functional requirements into subfunctions or subprocesses. This does not imply that the software design will also be partitioned that way.

! INFO

Use this structure to guide your team's requirements section. Your focus should be on what the system must do, not how it will be implemented.

1.5 Design Prototyping and Wireframes

Wireframes are **early-stage visual mockups** that illustrate layout, user interaction, and data flow within a website or application. They are not meant to be final designs, but rather tools to help your team:

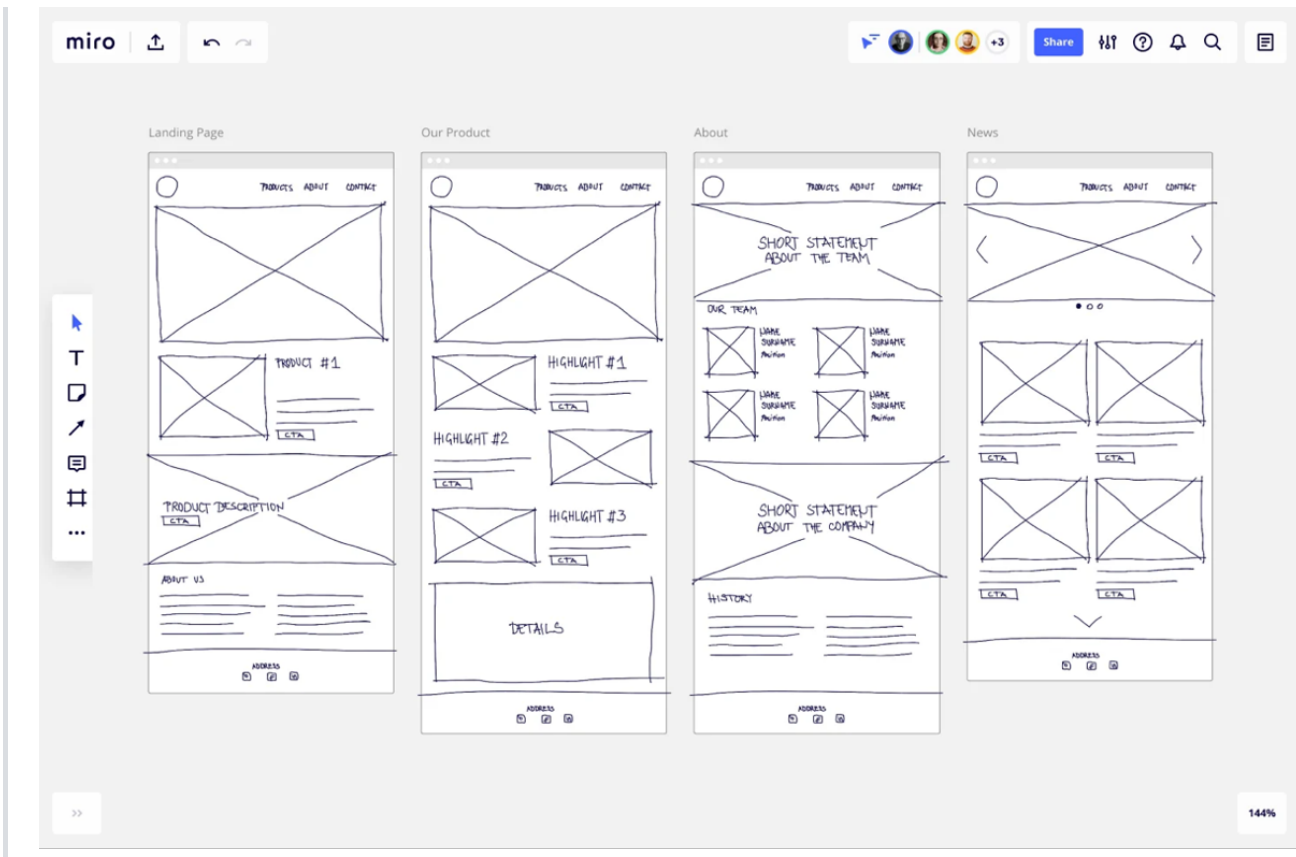
- Align on layout and structure
- Plan component interactions
- Guide front-end development
- Avoid major design changes later

A **good wireframe is not about artistic quality or detailed styling**. What matters most is that it is:

- **Consistent** – Layout, navigation, and structure should follow a logical pattern
- **Interactive** – Pages/components should reflect how users will interact with the system
- **Developer-friendly** – The wireframe should be understandable even for team members who didn't design it

Even low-fidelity sketches can be highly effective, as shown below:





💡 COLOURFUL IS NOT NECESSARY, BUT A DEFINED SCHEME HELPS

This example shows clearly labelled sections, repeating layout patterns, and a user flow that connects across pages. While bright colours are not necessary, using a **defined colour scheme** early on can help enforce consistency and guide future design decisions.

Choosing a Prototyping Tool

You may use any tool that helps your team collaborate and iterate quickly. Some recommended options include: [Figma](#), [Miro](#), [Lucidchart](#).

💡 HOW TO CHOOSE A PROTOTYPING TOOL

There's no need to choose a fancy or complex tool. Instead, select one that:

- Enables rapid prototyping
- Allows easy changes and flexibility
- Supports multiple design ideas
- Can be used without requiring advanced design skills

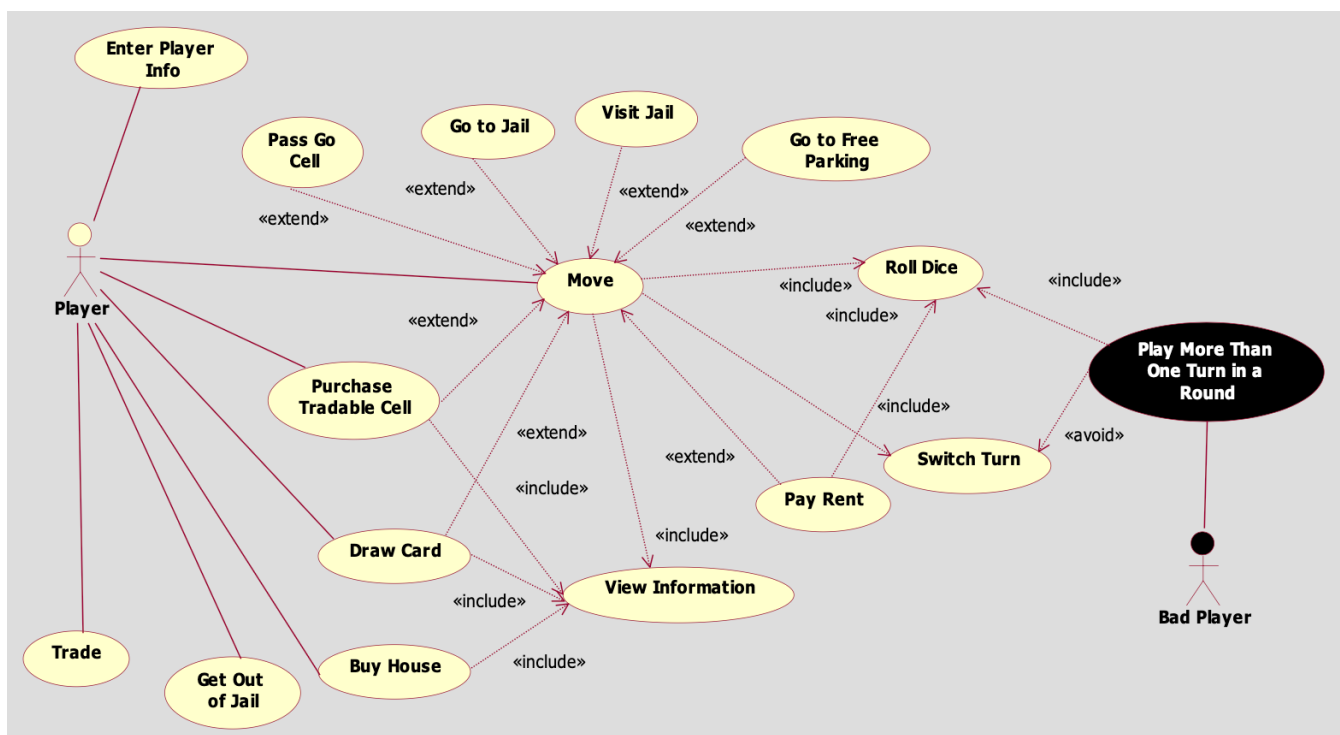
Later in the project, we'll revisit your prototype to evaluate **UI consistency, component relationships**, and how well the wireframe maps to your final implementation.

1.6 Use Case-Based Requirements (Optional Reading)

Use cases describe system behaviour from a user's point of view.

- Focused on **goal-oriented interaction**
- Can be written as **textual scenarios** or illustrated via **use case diagrams**
- Helps connect user intent to system features

This approach is useful for teams that want to go beyond static functional lists and explore how users interact with the system in context.

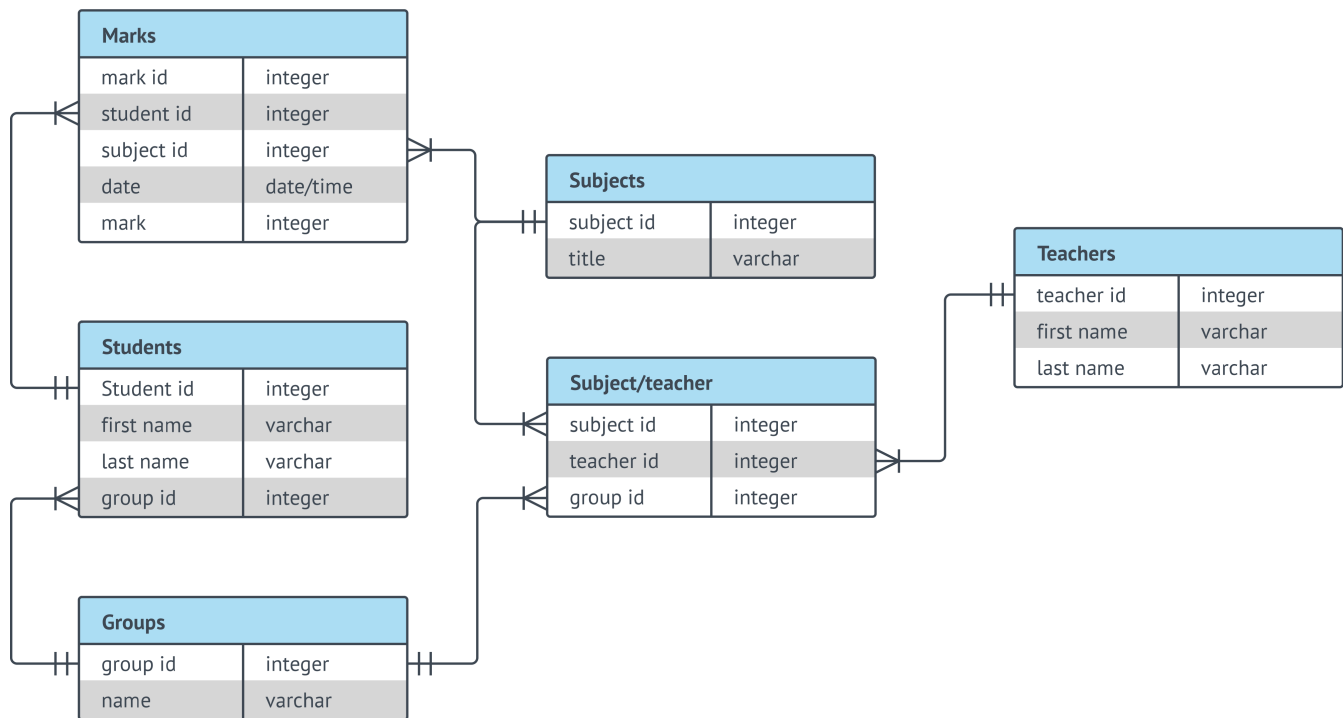


! INFO

The full use case documentation and description for this Monopoly Game can be found [here](#).

Chapter 2. Entity Relationship Diagram (ERD)

2.1 ERD Overview and Components



An **Entity Relationship Diagram (ERD)** models the structure of your database by defining the entities in your system, the relationships between them, and the data associated with each.

The table below outlines the four key components of an ERD:

Component	Description
Entity	These are the core concepts or objects in your system, e.g., <code>User</code> , <code>Product</code> , <code>Booking</code> .
Attribute	Properties or fields of an entity, e.g., a <code>User</code> might have <code>user_id</code> , <code>email</code> , and <code>created_at</code> . Attributes are usually listed inside the entity box.
Relationship	Indicates how entities are connected or interact, e.g., a <code>User</code> books a <code>Product</code> . Displayed as lines between entities.
Cardinality	Specifies how many instances of one entity relate to another, e.g., one-to-one, one-to-many, or many-to-many. These are marked at the ends of relationship lines.

NOTE

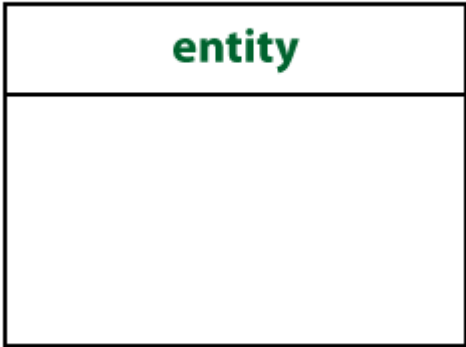
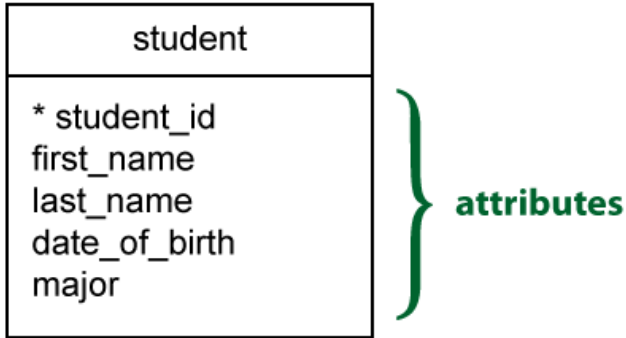
ERDs help communicate both **data structure** and **business rules** clearly, especially

during early system design.

2.2 Crow's Foot Notation

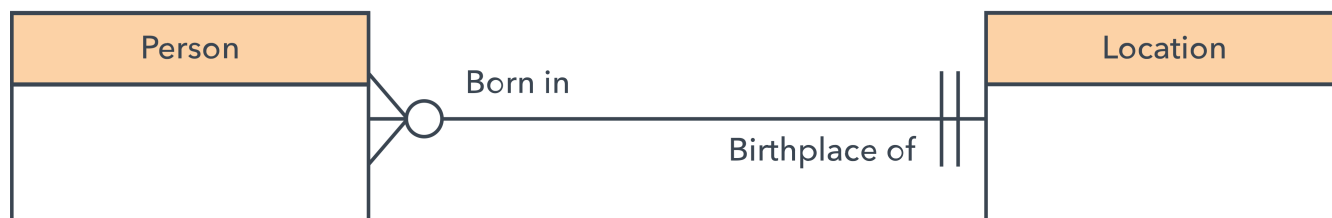
In assignment, we will use **Crow's Foot Notation** to draw ER diagrams. It visually distinguishes the **cardinality** and **optionality** of relationships, while preserving a clear structure.

Entity and Attribute

Entity	Attribute
Drawn as a rectangle with the entity name as a label.	Shown as listed fields inside the entity box.
Attributes are listed within the rectangle.	Use * to indicate primary keys.
Primary keys are often prefixed with an asterisk *.	Ensure attributes are atomic and meaningful.
	

Relationship

- Represented by a **line** connecting two entities.
- A **verb phrase** (e.g., *places, belongs to, includes*) should be used to describe the interaction.
- Relationships may be bidirectional or recursive when needed.

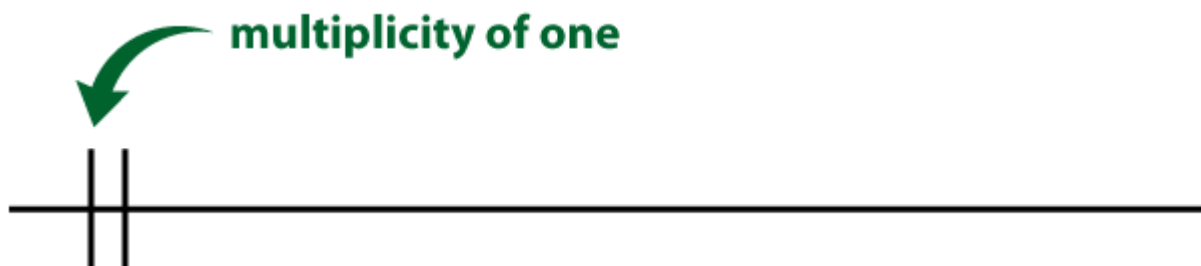


Cardinality

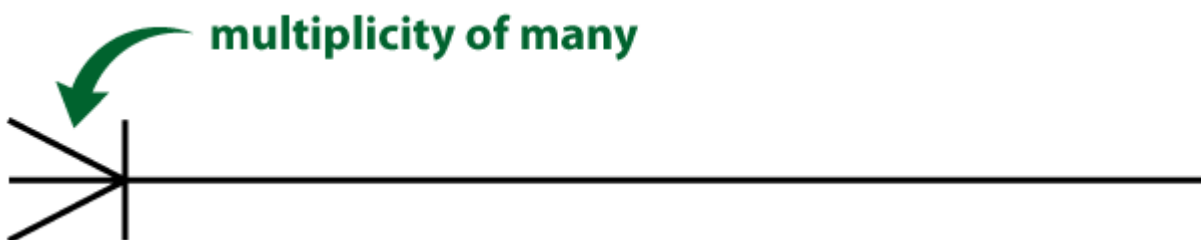
Cardinality defines how many instances of one entity can relate to another and whether that relationship is required. It is shown at **both ends** of the connecting line. **Cardinality consists of:**

► Multiplicity

- **Multiplicity of One** means exactly one instance is allowed.



- **Multiplicity of Many** means multiple instances are allowed.

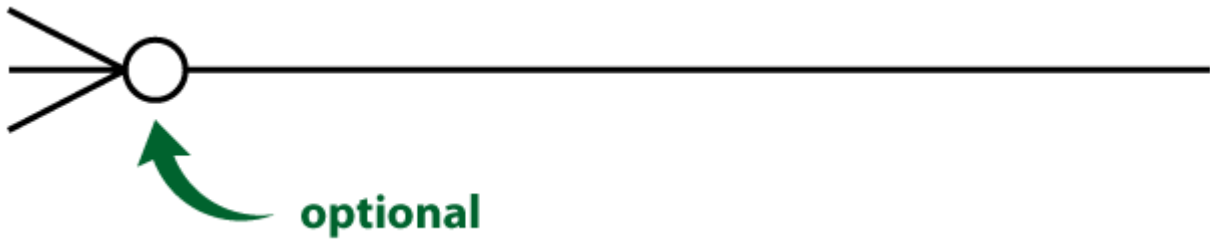


► Optionality

- **Mandatory** means the relationship is required (minimum = 1).



- **Optional** means the relationship is not required (minimum = 0).

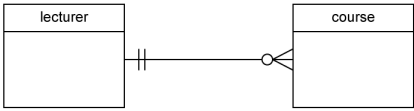
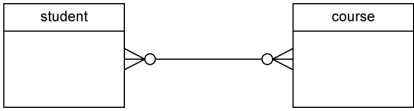


✚ Combined Cardinalities

Crow's Foot Notation combines multiplicity and optionality into four common patterns, shown below:

Relationship Type	Meaning	Example Notation Image
Zero or Many	Optional, any number	
One or Many	At least one	
One and Only One	Exactly one (mandatory)	
Zero or One	Optional, at most one	

Common Relationship Types

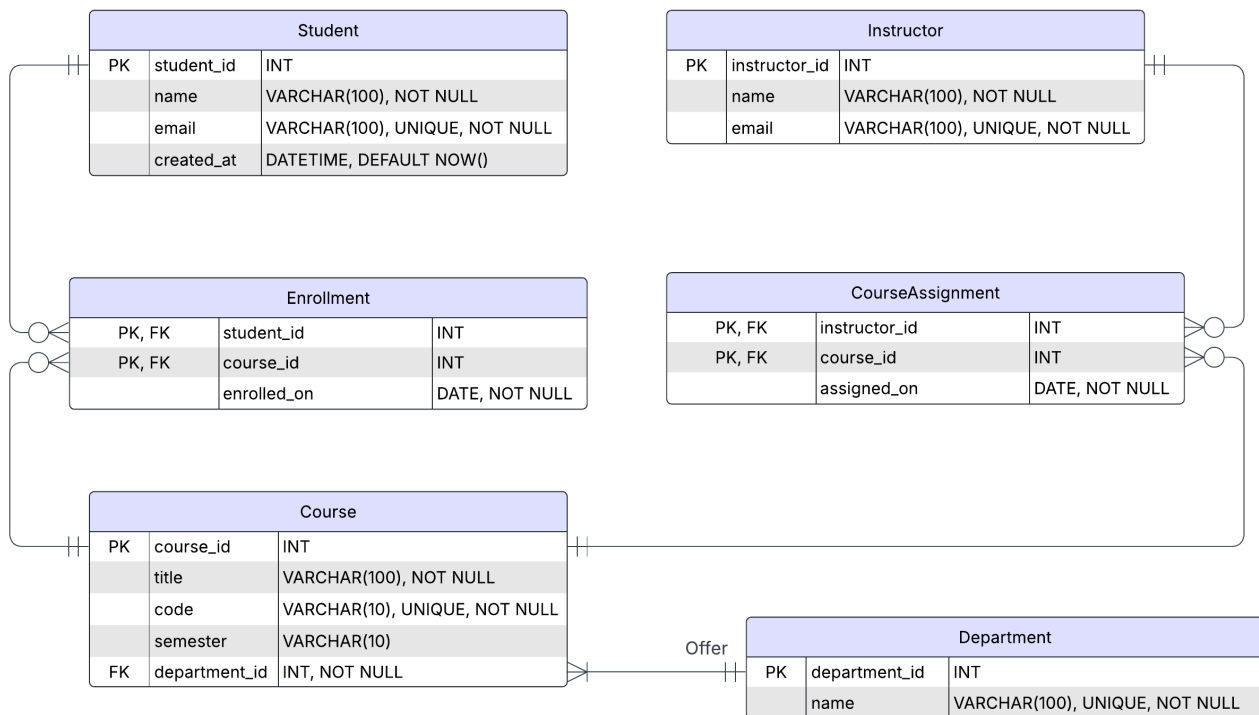
Diagram	Explanation
	One-to-One: Each student is assigned exactly one seat, and each seat is assigned to exactly one student. Both ends are mandatory, every student must have a seat, and every seat must be assigned.
	One-to-Many: One lecturer can teach many courses, but each course is taught by only one lecturer. A lecturer may teach zero courses, the relationship from lecturer to course is optional.
	Many-to-Many: Each student can enroll in many courses, and each course can have many students. However, a student may enroll in zero courses, and a course may have no students, both sides are optional.



BE CAREFUL WITH OPTIONALITY

Be careful with optionality when designing your project. Incorrect use of optional vs. mandatory relationships is a common mistake.

2.3 ERD Example in Crow's Foot Notation



The above Entity-Relationship Diagram (ERD) illustrates a well-normalised database design using Crow's Foot notation. It includes two **many-to-many** relationships:

- A student can enroll in many courses, and each course can have many students.
- An instructor can be assigned to multiple courses, and a course can have multiple instructors.

To achieve a normalised structure, instead of using many-to-many relationships directly, we introduce two junction tables:

- Enrollment connects students to courses.
- CourseAssignment connects instructors to courses.

Both junction tables include additional attributes (e.g., enrolled_on, assigned_on) and use composite primary keys, allowing flexibility and ensuring data integrity.



TIP

While Crow's Foot notation primarily focuses on entity relationships, this ERD also includes attribute data types and constraints (e.g., INT, VARCHAR, NOT NULL, UNIQUE). These details are not strictly required for ER diagrams, but including them improves clarity and communicates your design decisions more effectively, especially helpful when preparing your database for implementation.

Chapter 3. Database Quality and Project Setup

3.1 Database Design Recap (What to Watch For)

In this lab, we briefly revisit key principles of relational database design, not to teach database theory, but to help you **avoid common design mistakes** observed in past projects.

A well-designed schema should be **normalised**, **easy to extend**, and **aligned with real-world logic**. Below are three critical areas to keep in mind.

Tables and Entity Structure

- Make sure each table/entity represents a **single real-world concept** (e.g., User, Event, Booking).
- Avoid merging unrelated concepts into one table.
- Don't forget essential entities, think through your app's domain before you design.

! INFO

Example mistake: Combining users and admins into one table with a role flag, then hardcoding access logic instead of separating concerns properly. For example:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL UNIQUE,  
  password_hash TEXT NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  role VARCHAR(10) CHECK (role IN ('user', 'admin')) NOT NULL  
);
```

In the app logic, you often see something like this:

```
if user.role == 'admin':  
    show_admin_panel()  
else:  
    show_user_dashboard()
```

This approach works for small apps but quickly leads to tightly coupled roles and permissions, hard-to-maintain branching logic, and poor separation of concerns. It also makes it difficult to support more complex role-based behaviour later on.

Keys and Relationships

- Every table should have a **primary key** (PK).
- Use **foreign keys** (FKs) to establish relationships, don't rely on implicit naming.
- Use **junction tables** for many-to-many relationships. Each junction table should have two FKs and a **composite primary key**.

Fields, Types, and Constraints

- Use **appropriate data types** (e.g., `varchar(255)` for email, not everything).
- Define **NOT NULL**, `UNIQUE`, and other constraints clearly.
- Avoid vague or incomplete attribute lists, every table should store enough information to support your features.

Quick Design Check

Before moving on, make sure your database:

- Has no duplicated or redundant data
- Has all important entities and relationships represented
- Uses keys and constraints appropriately
- Can be explained to others (e.g., via an ERD) clearly and confidently



TIP

You'll revisit your design again before implementation, but getting the core structure right early will save you trouble later.

Quick Refresher: Normalisation (1NF to BCNF)

Relational database design should follow **normalisation principles** to reduce redundancy and improve data integrity. Below is a summary of key normal forms you are expected to apply in your design:



Normal Form	Description
1NF (First Normal Form)	All fields should contain atomic values . No repeating groups or multi-valued attributes.
2NF (Second Normal Form)	All non-key attributes must be fully dependent on the entire primary key . Especially important for composite keys.
3NF (Third Normal Form)	There should be no transitive dependencies , non-key attributes must not depend on other non-key attributes.
BCNF (Boyce-Codd Normal Form)	A stricter version of 3NF. Every determinant must be a candidate key.

WARNING

Many-to-many relationships stored directly in one table (e.g., lists or repeated fields) are not in 1NF. Use proper **junction tables** with composite primary keys to resolve them.

3.2 Project Setup and Structure Guidelines

To prepare for scalable and maintainable development, your team should define and follow a clear set of technical conventions.

Organise Your Project

- Structure folders clearly (e.g., `backend/`, `frontend/`, `docs/`)
- Use virtual environments and `.env` files
- Track dependencies:
 - `requirements.txt` (Python/Django)
 - `package.json` (JavaScript/React)

Use Git Properly

- Set up Git with clear commit messages
- Protect main branches (via GitHub settings) and define rulesets
- Require pull requests and code reviews



Testing and Code Quality

- Include integration testing and unit testing
- Use test runners (`pytest`, `npm test`)
- Track passing status and coverage

Style Guides and Standards

- Follow PEP 8, Django conventions
- Use Prettier or ESLint for frontend
- Maintain clean import structure and consistent formatting

Documentation

- Maintain a `README.md` with setup, team details, and contribution workflow
- Document API endpoints using Postman, Swagger, or plain Markdown
- Add in-code comments and developer notes when necessary

Consistent setup and documentation improves onboarding, collaboration, and long-term maintainability.

Further Readings

- [What is an Entity Relationship Diagram \(ERD\)?](#)
- [PEP 8 – Style Guide for Python Code](#)

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.

